

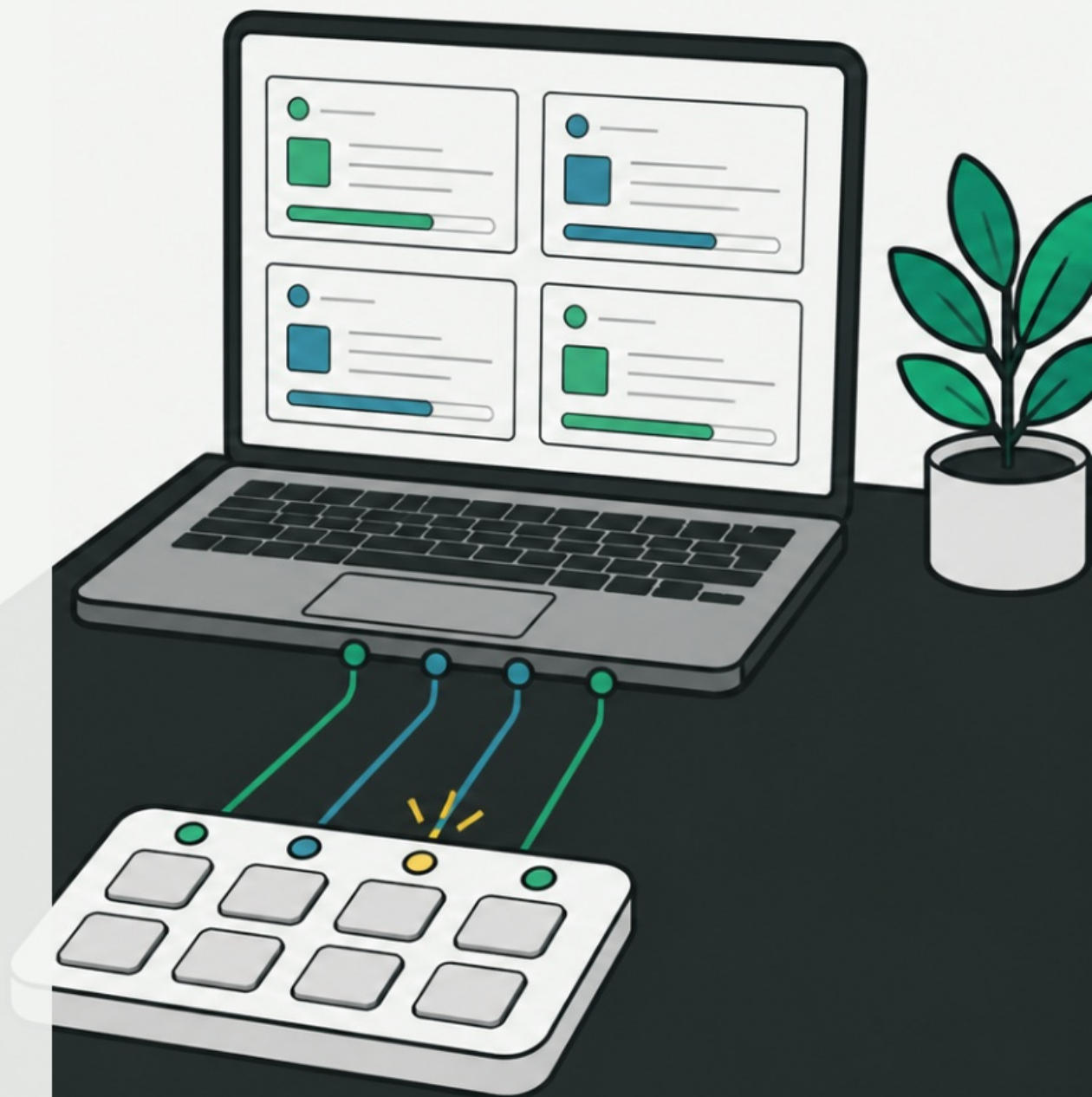
Codex 任务电台

一把键盘，远程开发

Lark

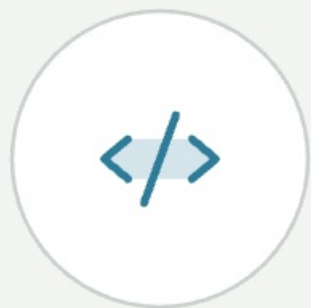
知名外企嵌入式软件与 AI 产品转型负责人

EasyInput V2 × Codex × 本地语音



Lark: 从嵌入式软件到 AI 产品

这不是一次“换赛道”，而是把原来的工程经验接到新的工具上。



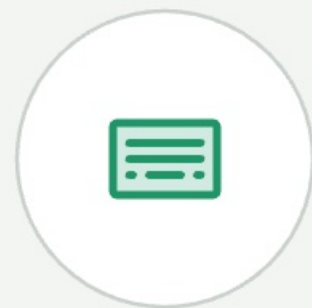
嵌入式软件

硬件、固件、系统边界



AI 工具实践

把模型放进真实工作流程



AI 产品

让人真的愿意每天用

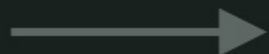
20 秒看懂它

真实演示只有四个动作。



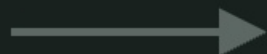
说

按住上排键



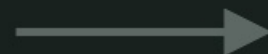
做

Codex 在后台



亮

有新汇报



听

按一下下排键

把四个 Codex 任务，
变成四个可说、可听、会亮灯的实体频道。



不是多做一个屏幕，而是少看一点屏幕。

我每天同时开着好几个 **Codex** 任务

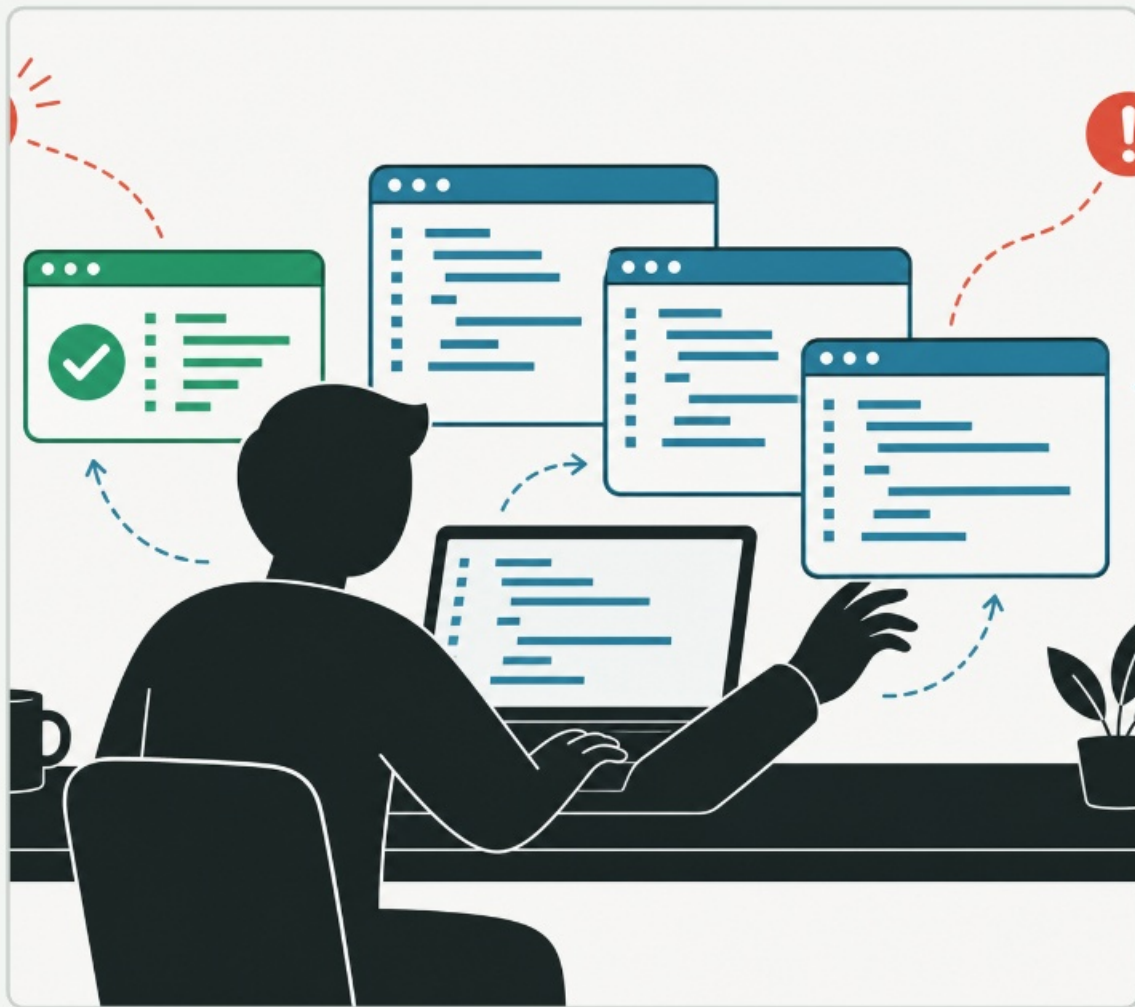
一个在改固件

一个在做 Mac App

一个在跑测试

还有一个在整理课程

四个任务都能跑，
但我一直在来回看。



麻烦不是 AI 不够快，是我一直在打断自己



总在打字

很多要求只要说一句，
却要先坐回电脑前。



总在切窗口

看谁做完了，
看谁还在等我。

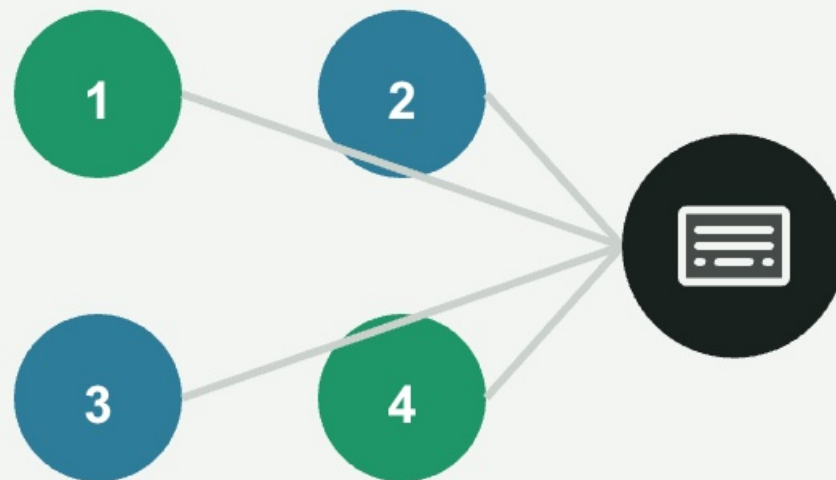


总在等结果

盯着进度，
反而没去做别的事。

AI 能并行， 人却只有一双手。

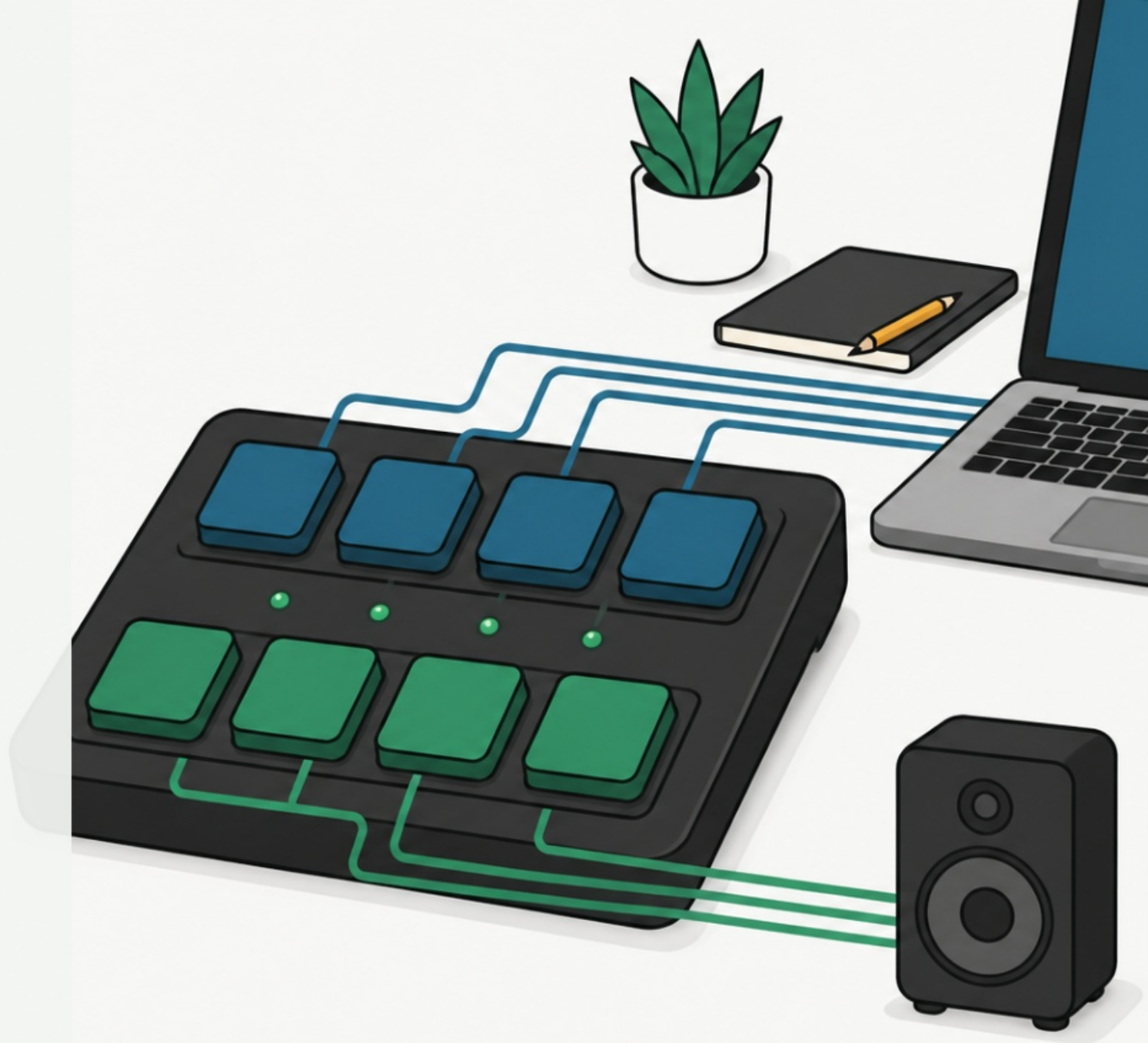
真正卡住的，是下要求、看状态、收结果。



手边正好有一块 八键开发板

八个键、五颗灯、麦克风、
Wi-Fi，还有扬声器链路。

不如把它改成任务对讲机。



四个任务，就是四个频道

我不需要一直看它，只要知道该对哪个频道说、听哪个频道。



CH 1

固件任务



CH 2

App 任务



CH 3

测试任务



CH 4

课程任务

最初的交互草图，最后基本没变



S1-S4
按住说要求

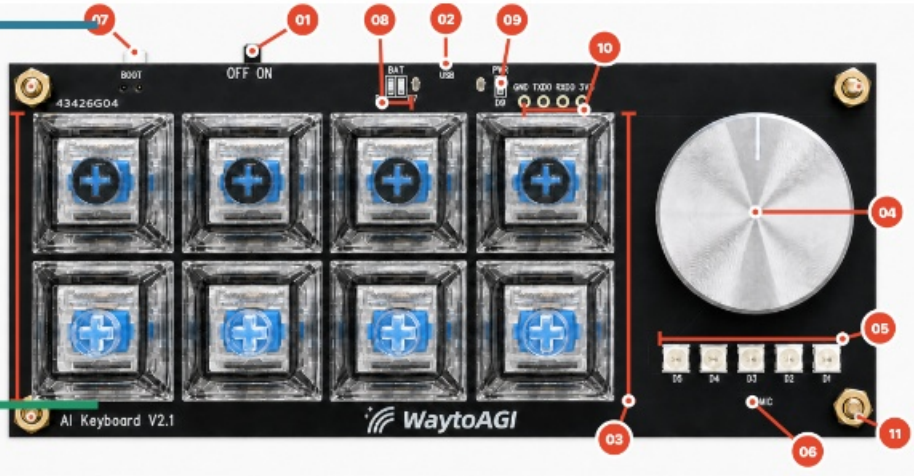


S5-S8
按一下听汇报

EASYINPUT · FRONT COMPONENT GUIDE

认识开发板正面

从日常操作，到状态、恢复与调试入口



第一次使用先认识 01-06。BOOT 与串口触点只在烧录、恢复或进阶调试时使用；BOOT 不是蓝牙配对键。

EASYINPUT V2.0 - ASSEMBLED FRONT VIEW

01-11

COMPONENT INDEX

正面部件索引

日常操作与连接

- 01 电源开关
拨到 ON 开机，OFF 关机
- 02 USB-C 接口
供电、数据连接与固件烧录
- 03 机械按键 S1-S8
八个可配置的实体输入
- 04 旋钮 S9
支持旋转、短按与长按
- 05 RGB 彩色灯 D1-D5
灯光反馈、短按与长按反馈
- 06 麦克风收音孔
使用键盘麦克风时请前遮挡
- 状态、恢复与调试
- 07 BOOT 下载 / 恢复键
固件烧录或设备恢复时使用
- 08 充电状态灯 D6 / D7
显示充电相关状态
- 09 独立状态灯 D9
硬件已保，当前固件尚未应用
- 10 串口调试触点
GND / TXD0 / RXD0 / 3V3

安装结构

- 11 四角固定柱
用于安装原型或外壳



左四灯 = 信箱
最右灯 = 运行数



旋转：逆时针减 · 顺时针加（板载扬声器）
短按：播报当前音量

EasyInput 不是一把固定功能的键盘

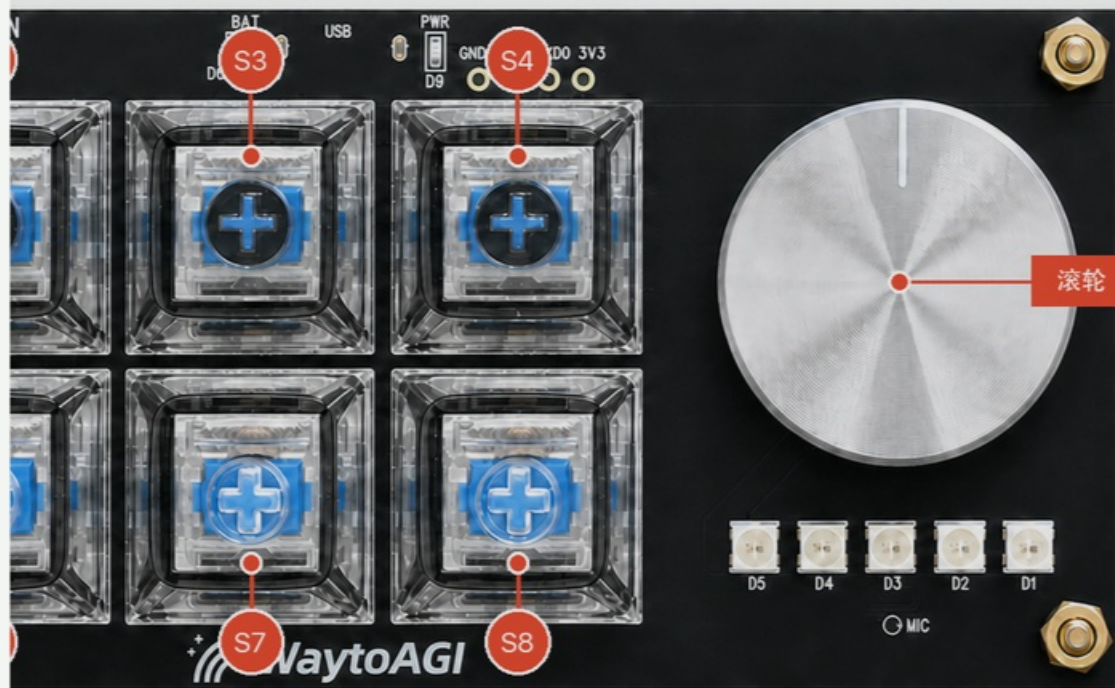
它的本质是一块 AI 硬件开发板。

键位只是一个起点。

你可以重新定义它与电脑的关系。

默认功能

先认清 S1 - S8，再按功能使用



DEFAULT

出厂

App 中的
板上的 S

S1 语

S2 回

S3 A

S4 退

S5 全

S6 复

S7 粘

S8 摘

滚轮

转动：

在 App 中

用人话认识这块硬件



能按

8 个机械键



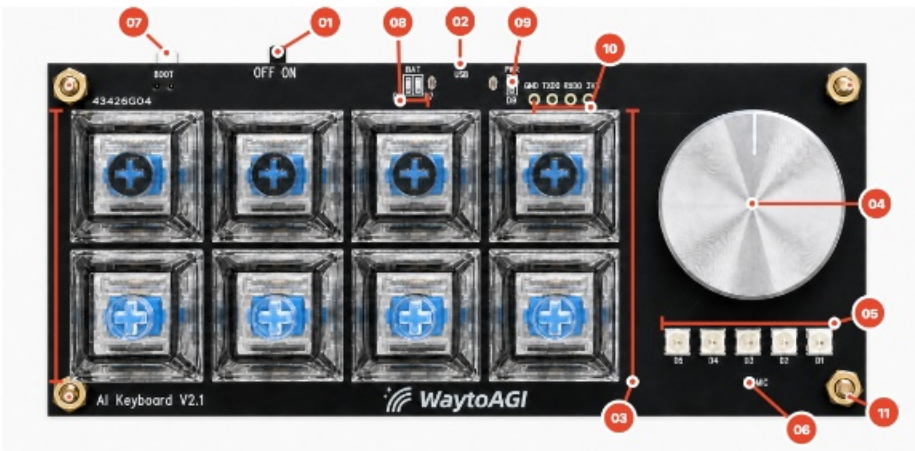
调板载音量

10 档 · 短按播报

EASYINPUT · FRONT COMPONENT GUIDE

认识开发板正面

从日常操作，到状态、恢复与调试入口



第一次使用先认识 01-06。BOOT 与串口触点只在烧录、恢复或进阶调试时使用；BOOT 不是蓝牙配对键。

EASYINPUT V2.0 · ASSEMBLED FRONT VIEW

01-11

COMPONENT INDEX

正面部件索引

日常操作与连接

01 电源开关
控制 ON 开机，OFF 关机

02 USB-C 接口
供电、数据连接与固件烧录

03 机械按键 S1-S8
八个可配置的功能键

04 旋钮 S9
关闭/唤醒、短按与长按

05 RGB 彩色灯 D1-D5
灯光反馈、动画与按键反馈

06 麦克风收音孔
使用按键麦克风时请勿遮挡

状态、恢复与调试

07 BOOT 下载 / 恢复键
固件烧录或设备恢复时使用

08 充电状态灯 D6 / D7
显示充电相关状态

09 独立状态灯 D9
硬件已接，当前固件未应用

10 串口调试触点
GND / TXD0 / RXD0 / SW0

安装结构

11 四角固定柱
用于安装亚克力外壳

FRONT COMPONENTS / 2026-07-24



能听

I2S 麦克风



能说

功放与扬声器



能亮

5 颗 WS2812

再加上 USB、蓝牙和 Wi-Fi，它就能和电脑一起工作。

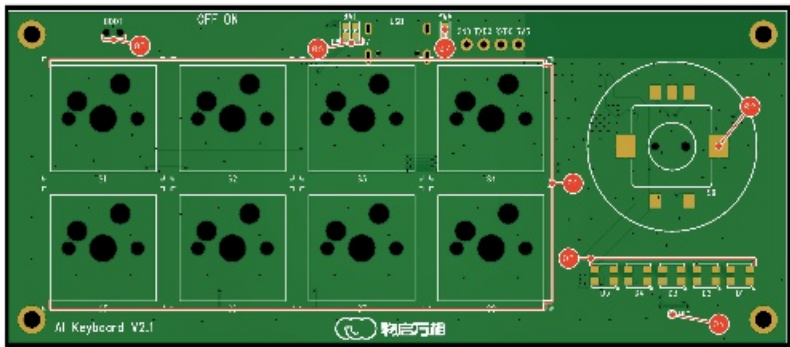
这次真正用上了四个原本闲着的能力

PCB 正面：按键、旋钮与状态反馈

本文档包含位置与尺寸信息，可以帮助你准确地安装按键、旋钮与状态反馈。

PCB 型号：M Keyboard V2.1

用户操作与反馈部件的安装位置



M Keyboard V2.1

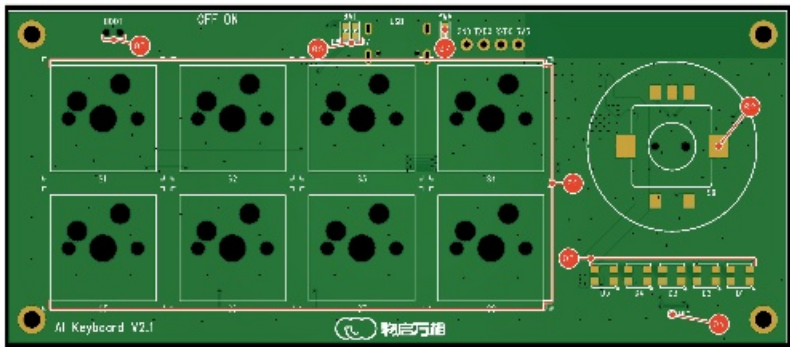
01

10%

本文档包含位置与尺寸信息，可以帮助你准确地安装按键、旋钮与状态反馈。

PCB 型号：M Keyboard V2.1

用户操作与反馈部件的安装位置



M Keyboard V2.1

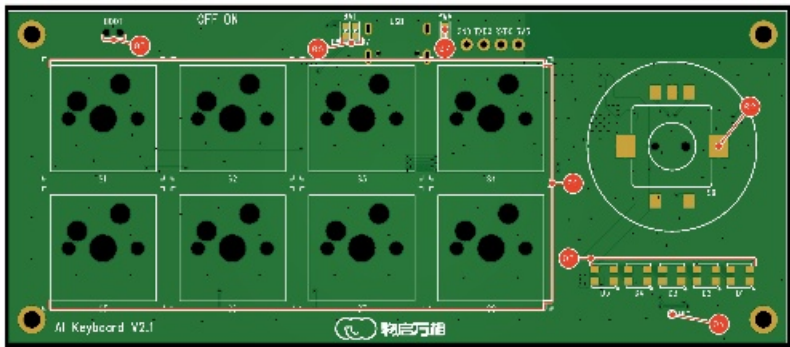
01

10%

本文档包含位置与尺寸信息，可以帮助你准确地安装按键、旋钮与状态反馈。

PCB 型号：M Keyboard V2.1

用户操作与反馈部件的安装位置



M Keyboard V2.1

01

10%

[illegible]

PSRAM

缓存录音

Wi-Fi 音频

把声音送到 Mac

数字功放

驱动播放

扬声器

把汇报说出来

第一版先把本地 **Wi-Fi** 跑通

键盘和 Mac 在同一个网络里，先把声音、状态和播放做扎实。



这里的“远程”是离开 **Codex** 窗口，不是已经完成公网遥控。

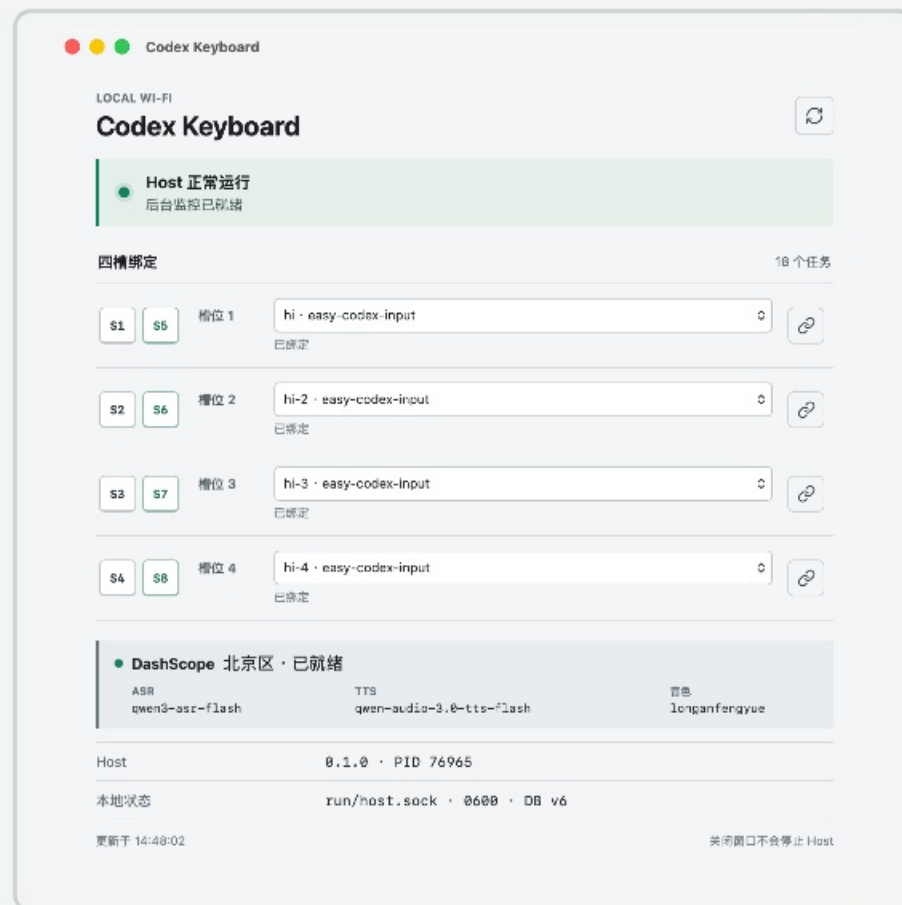
Mac App 负责绑定四个任务

真实界面截图，任务名称已换成课程示例。

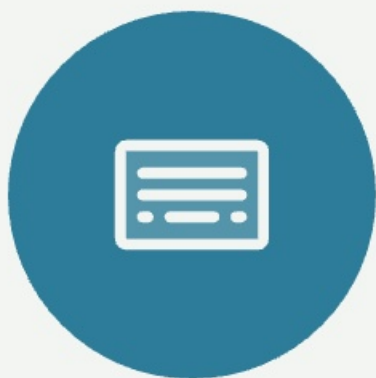
每个槽只做三件事

1. 选择一个 Codex 任务
2. 显示是否有未听汇报
3. 显示键盘是否在线

数据库只由 Host 管，
界面不直接碰底层状态。

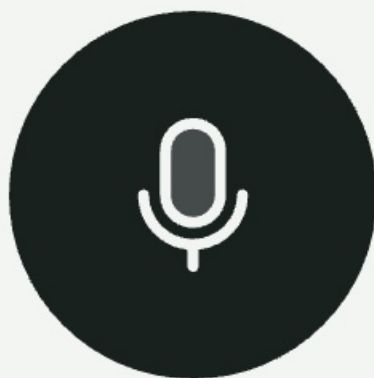
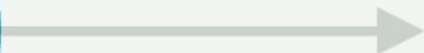


按住说，松开就发



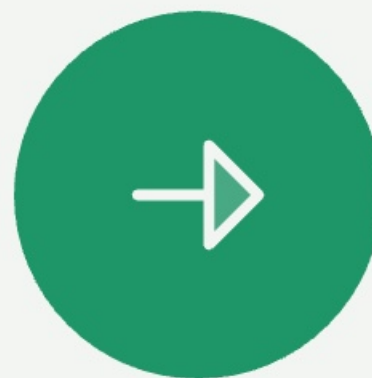
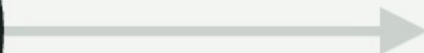
按住 S1-S4

选中对应任务



直接说要求

录音留在本地链路



松开后发送

ASR 结果进入 Codex

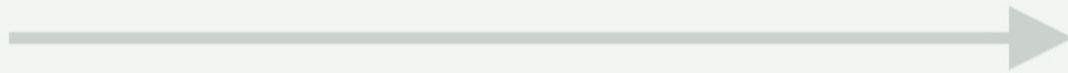
发完就走。

Codex 在后台干活。

这才是我真正想省下来的时间：不用守着屏幕。



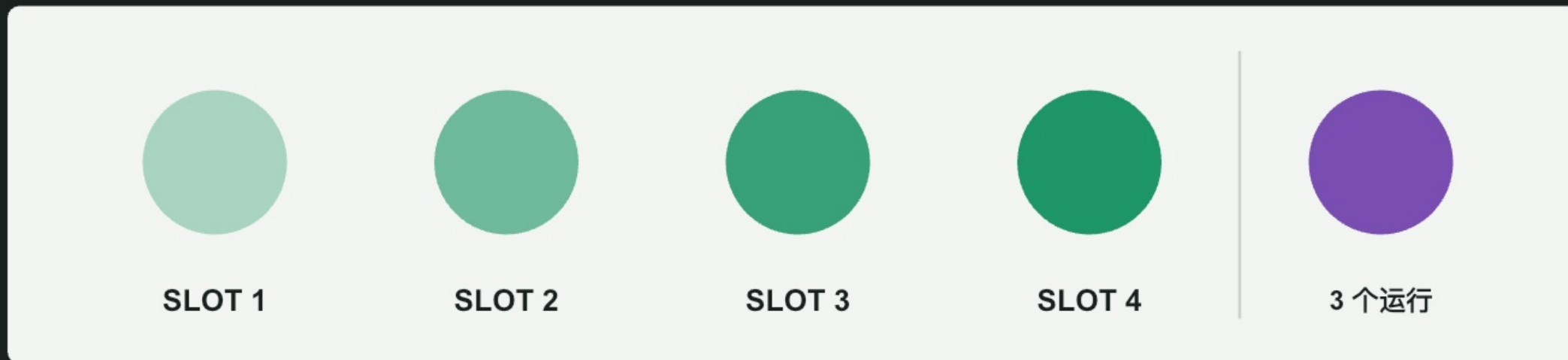
一句要求



一段完整工作

前四颗看信箱，第五颗看忙不忙

左起 1-4 对应四个槽；最右灯用颜色表示正在运行的任务数。



信箱灯 未听越多



第 5 灯



0



1



2



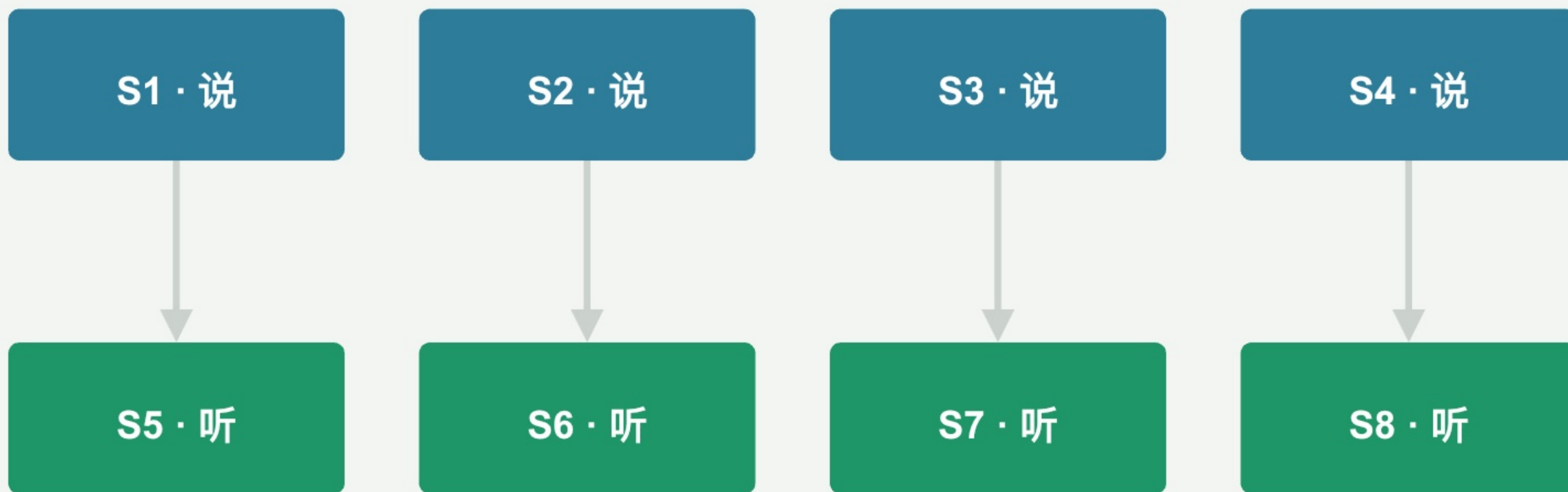
3



4

绿 / 黄 / 橙 / 紫 / 红
全部完成后回到绿色

S5-S8：按一下，听对应任务汇报

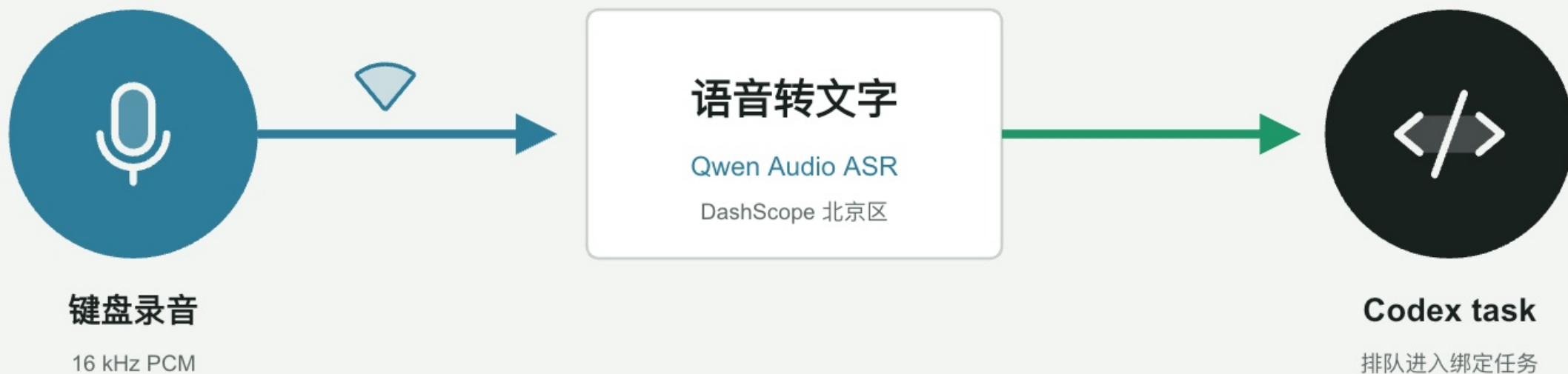


一组键位就是一个频道：上排发消息，下排收回信。

三个角色，分工很简单



说进去：麦克风 → 千问 **ASR** → **Codex**



键盘不理解内容。它只负责把声音可靠地送到 Mac。

做完后：Codex → Luna High 总结

只总结最近几次助手的最终回复，不把过程日志念给人听。



任务完成事件

拿到助手最后的答复

保留完成了什么

保留结果和待办



Codex Luna High 提炼

“四个任务已经并行完成，活动灯达到红色。
四槽总结都已生成，可以逐个收听。”

旧的未听内容，会一起重新整理进去。

Host 只守运行底线，不替 Luna 做内容评分。

听回来：千问 TTS → 本地缓存 → 扬声器



音频在 Mac 上缓存；键盘按下后，通过局域网流式取回。

真正播完，才算“听过”

把包传完，不代表声音已经从扬声器出来。



不能在这里熄灯

音频传输完成

扬声器可能还在播放

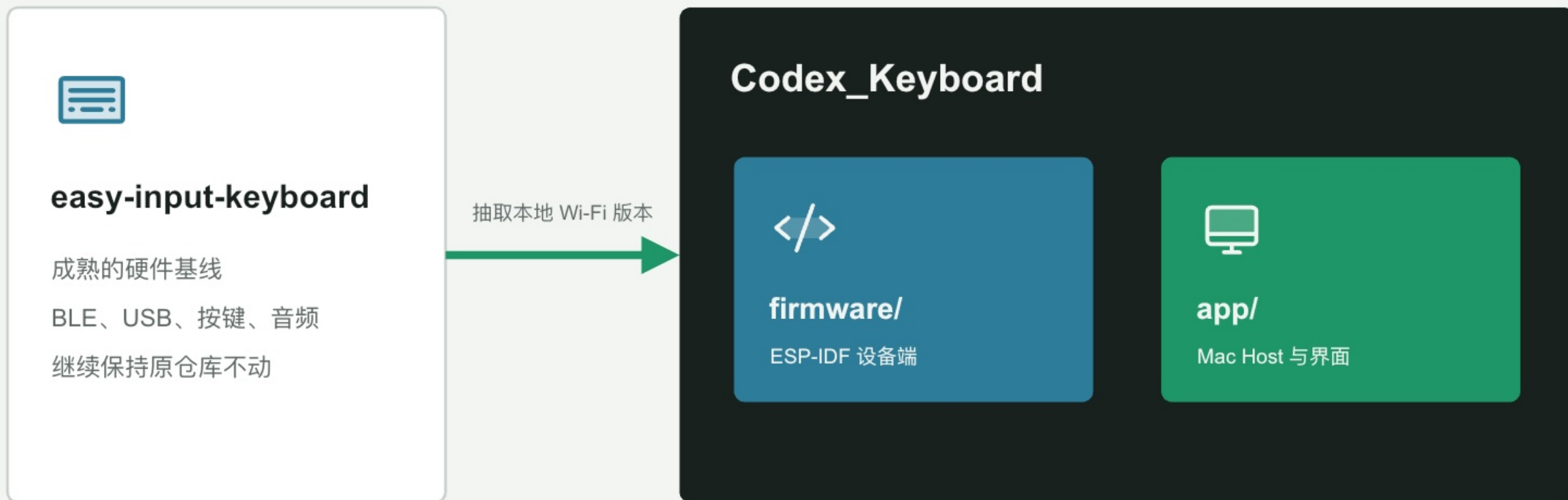


在这里才删除缓存

扬声器确认播放结束

灯熄灭，音频删除

从现有键盘，抽出自己的 **App** 和固件



第一次卡住：能烧录，不等于能播放



进不了下载模式

症状：串口只在上电瞬间出现。

原因：这块板的 BOOT 时序不稳定。

做法

先拉住 debug reset，再抓断电上电瞬间。



有开机音，却没有任务音频

这只证明扬声器和功放能响。

它不证明 UDP、缓存、解码都通。

做法

沿“请求 → 传输 → 解码 → 扬声器”逐段查。

更难的是那些“看起来差不多”的问题



每次都要等很久

UDP 窗口大过设备队列，
丢包后反复重传。

修：窗口与 lwIP 队列对齐



灯从右往左亮

代码里的第 1 颗，
不是 PCB 正面的最左灯。

修：按真实灯带链路反向映射



任务完成，S7 却不亮

总结内容其实可用，
却被过度门禁反复拒绝。

修：质量交给 Luna，只留运行底线

测试通过，不等于耳朵真的听到了



最后一级，只能靠真实设备和人的耳朵验收。

学员还能把它改成什么

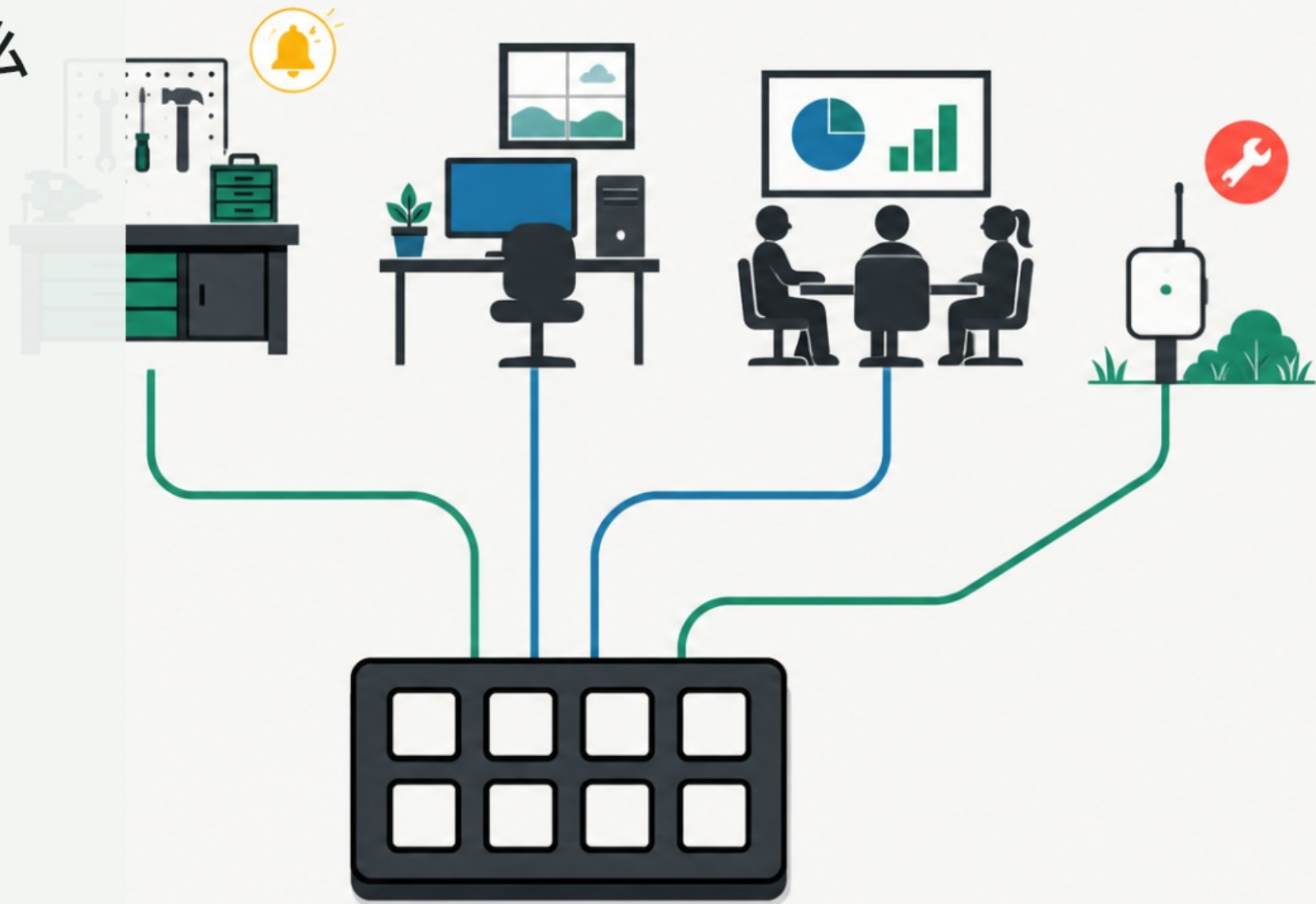
会议任务遥控器

工位语音信箱

设备调试台

家庭自动化控制台

先找一个每天都在重复的动作，
再决定它该长什么样。





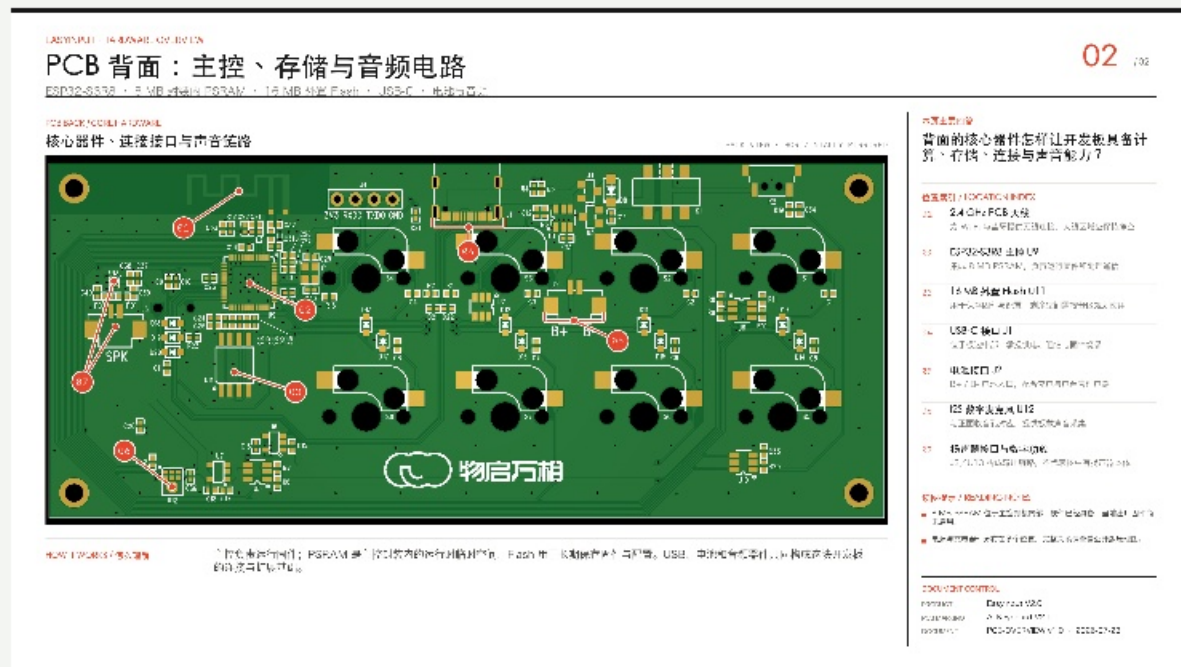
AI 硬件， 不是把模型塞进板子。

而是给 AI 一副能进入日常工作的身体。

它能听见你的要求，安静地等结果，
再用灯和声音把重要的事带回来。

Codex 任务电台 · Lark

附录：EasyInput V2 公开硬件参数



EasyInput V2 PCB 背面

ESP32-S3R8

8 MB PSRAM · 16 MB Flash

输入：8 键 + 旋钮旋转/按压

灯光：5 颗 WS2812

音频：I2S 麦克风 + 数字功放

连接: USB-C、BLE、2.4 GHz Wi-Fi

调试：BOOT 入口、UART 触点

板载十档音量、短按播报与扬声器均由自定义固件启用

附录：S1-S8 与 GPIO 对照

按 PCB 正面丝印，从左到右读取。

上排 · 语音输入

S1 · GPIO2

S2 · GPIO47

S3 · GPIO38

S4 · GPIO41

下排 · 收听汇报

S5 · GPIO1

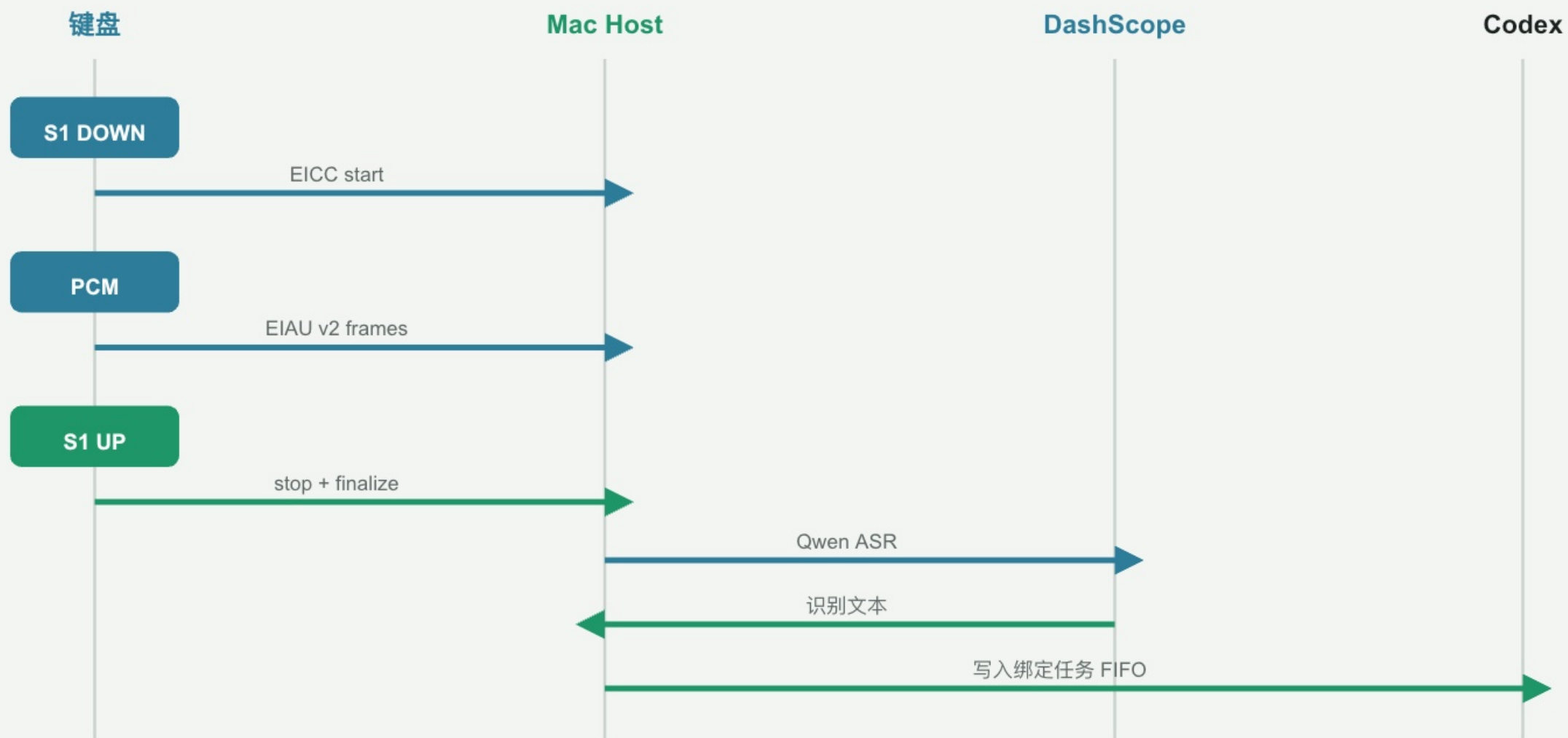
S6 · GPIO6

S7 · GPIO7

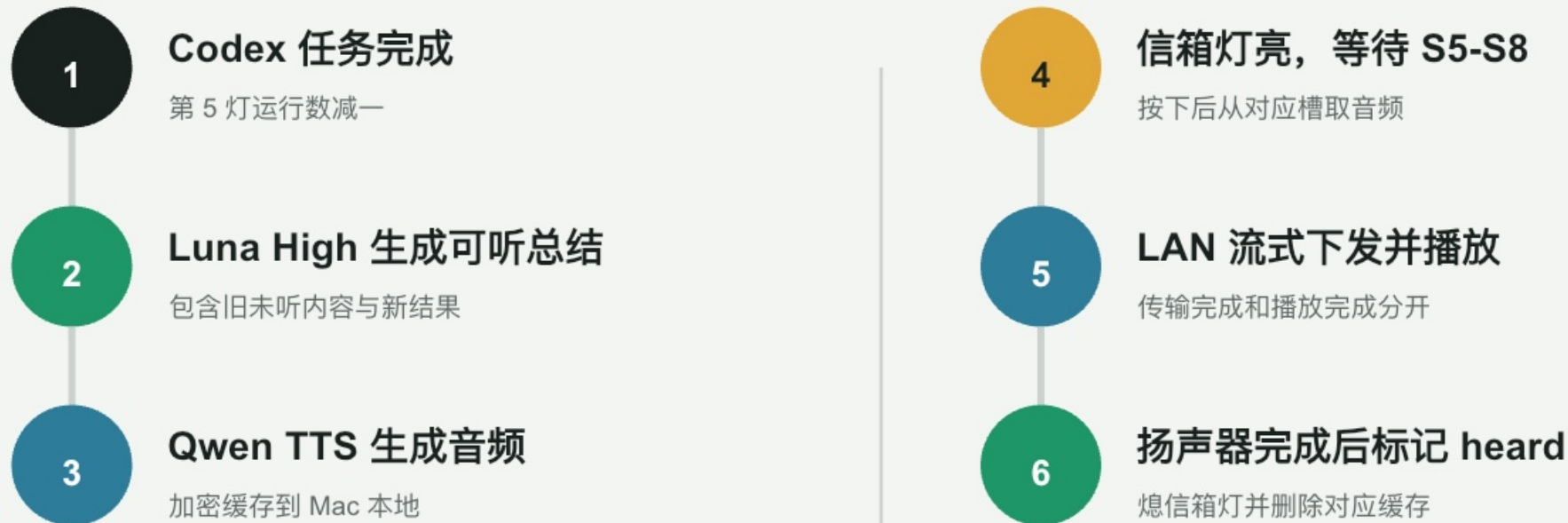
S8 · GPIO48

排查键位时先看丝印和运行诊断，不要根据最终功能反推 GPIO。

附录：完整语音输入时序



附录：总结、缓存与播放时序



所有任务完成后第 5 灯回绿；任何一步失败，仍保留未听内容。

附录：密钥、缓存和数据放在哪里



API Key

只在 Mac 本地 `.env`

文件权限 0600

绝不提交 Git



摘要与音频

Host 管理 SQLite

音频加密缓存到 Mac

听完后精确删除



键盘设备

只保存 Wi-Fi 与 Host 配置

运行时只认槽位号

不保存 Codex 对话

公开仓库只包含代码、文档和脱敏截图，不包含任何真实任务 ID 或密钥。

附录：仓库与测试入口



Larkspur-Wang / Codex_Keyboard

github.com/Larkspur-Wang/Codex_Keyboard

firmware/

ESP-IDF 主工程
host_test / CTest
真机烧录与 HIL

app/

Rust Host
Dashboard UI
SQLite / LAN / TTS

docs/

产品规格
架构与协议
开发过程记录

建议从 README 的同 Wi-Fi 演示路径开始，再看协议与测试。